Université Bretagne Sud
IUT de Vannes
Département Informatique — 2026/2027

BUT 2 — R3.07. SQL dans un langage de programmation

TD/TP 3

Index et optimisation de requêtes

1 Objectifs et organisation

Ce TP met en pratique le chapitre 3. Il dure environ **une heure** (après 30 minutes de cours). Vous allez :

— lire des plans d'exécution avec `EXPLAIN ANALYZE` ;
— créer des index et **mesurer** leur effet, en lecture comme en écriture ;
— comprendre pourquoi un index n'est parfois pas utilisé ;
— optimiser des insertions depuis Java avec les lots (*batch*).

> **Ce qu'on attend**
>
> Pour chaque exercice, notez dans un fichier `OPTIMISATION.md` de votre dépôt : la requête, le type de parcours lu dans le plan (*Seq Scan*, *Index Scan*…) et le temps d'exécution **avant** et **après** votre modification. Les temps varient d'une machine à l'autre : c'est l'**ordre de grandeur** qui compte.

2 Mise en place

Le script `code/generer_donnees.sql` (fourni sur Moodle) crée un schéma séparé `perf` avec 100 000 clients, 200 000 comptes et 1 000 000 d'opérations. Il ne touche pas à vos tables des TP précédents. **Seules les clés primaires sont indexées.**

```
docker cp generer_donnees.sql <conteneur>:/tmp/
docker exec -it <conteneur> psql -U user -d banque_db -f /tmp/generer_donnees.sql
docker exec -it <conteneur> psql -U user -d banque_db
```

Dans `psql`, placez-vous dans le schéma et affichez le temps des requêtes :

```
SET search_path TO perf;
\timing on
\d Operation
```

3 Lire un plan d'exécution

Exercice 1 — Premiers plans

a) Exécutez `EXPLAIN ANALYZE` sur les deux requêtes suivantes :

```sql
SELECT * FROM Operation WHERE numCompte = 4242;

SELECT c.numClient, sum(o.montant)
  FROM Compte c JOIN Operation o ON o.numCompte = c.numCompte
 WHERE c.numClient = 4242
 GROUP BY c.numClient;
```

b) Pour chacune : quel type de parcours est utilisé sur `Operation` ? Combien de lignes sont lues puis rejetées (`Rows Removed by Filter`) ? Combien sont finalement retournées ?

c) Comparez `EXPLAIN` et `EXPLAIN ANALYZE` sur la première requête. Quelle est la différence ?

4 Indexer les clés étrangères

Exercice 2 — Index sur les clés étrangères

a) Les colonnes `Operation.numCompte` et `Compte.numClient` sont des clés étrangères. Sont-elles indexées ? (`\di` ou `SELECT * FROM pg_indexes WHERE schemaname = 'perf';`)

b) Mesurez la suppression de 50 comptes, **dans une transaction annulée** pour ne rien perdre :

```sql
BEGIN;
DELETE FROM Operation WHERE numCompte BETWEEN 1 AND 50;
EXPLAIN ANALYZE DELETE FROM Compte WHERE numCompte BETWEEN 1 AND 50;
ROLLBACK;
```

Pourquoi est-ce si long alors qu'on ne supprime que 50 lignes ? (Indice : que doit vérifier la base pour chaque compte supprimé ?)

c) Créez les deux index :

```sql
CREATE INDEX idx_operation_compte ON Operation(numCompte);
CREATE INDEX idx_compte_client    ON Compte(numClient);
```

puis refaites les mesures des questions précédentes. Quels types de parcours apparaissent maintenant ? Quel est le facteur de gain ?

5 Quand l'index n'est pas utilisé

Exercice 3 — Fonctions, `LIKE` et index composites

a) Créez `CREATE INDEX idx_client_nom ON Client(nomClient);` puis analysez :

```sql
SELECT * FROM Client WHERE nomClient = 'Nom4242';
SELECT * FROM Client WHERE lower(nomClient) = 'nom4242';
SELECT * FROM Client WHERE nomClient LIKE 'Nom424%';
```

Lesquelles utilisent l'index ? Proposez et créez un index pour chacune des deux autres (voir le chapitre 3), puis vérifiez.

b) Créez `CREATE INDEX idx_op_compte_date ON Operation(numCompte,`

`dateOperation);`. Pour chacune des requêtes suivantes, prédisez d'abord si l'index sera utilisé, puis vérifiez :

```sql
SELECT * FROM Operation WHERE numCompte = 42 AND dateOperation >= '2024-01-01';
SELECT * FROM Operation WHERE dateOperation = '2022-06-01';
```

c) Créez un index sur `typeOperation` et analysez `SELECT * FROM Operation WHERE typeOperation = 'DEPOT';`. Quelle proportion des lignes est retournée ? Le gain est-il comparable à celui de l'exercice 2 ? Pourquoi ?

Exercice 4 — Le coût des index

a) Créez une copie vide de la table, sans index : `CREATE TABLE op_copie (LIKE Operation);`

b) Mesurez (`\timing`) : `INSERT INTO op_copie SELECT * FROM Operation LIMIT 300000;`

c) Videz la table (`TRUNCATE op_copie;`), créez-y 4 index (sur `numCompte`, `typeOperation`, `dateOperation` et `(numCompte, dateOperation)`) et refaites la même insertion.

d) Concluez : faut-il indexer « toutes les colonnes, au cas où » ?

6 Optimiser côté Java : les lots

Dans votre projet du TP2, ajoutez une classe `BenchInsertions` qui insère **10 000 opérations** dans `perf.Operation` (numéros à partir de 2 000 001, `numCompte` entre 1 et 200 000) et affiche la durée de chaque variante mesurée avec `System.nanoTime()`. Supprimez les lignes insérées entre deux variantes (`DELETE FROM perf.Operation WHERE numOperation > 2000000`).

Exercice 5 — Une ligne à la fois ou par lots ?

a) **Variante 1** : un `PreparedStatement`, `executeUpdate()` pour chaque ligne, en `autoCommit = true`.

b) **Variante 2** : même chose, mais dans **une seule transaction** (`setAutoCommit(false)` puis `commit()`).

c) **Variante 3** : une seule transaction et `addBatch()` / `executeBatch()` par paquets de 1000.

d) **Variante 4** : variante 3 en ajoutant `?reWriteBatchedInserts=true` à l'URL JDBC.

e) Présentez les quatre durées dans un tableau et expliquez les écarts (allers-retours réseau, écritures disque au `COMMIT`).

```java
String url = "jdbc:postgresql://localhost:5432/banque_db";   // variante 4 : + "?reWriteBatchedInserts=true"
String sql = "INSERT INTO perf.Operation VALUES (?, ?, ?, ?, ?)";
long debut = System.nanoTime();
// ... insertions ...
System.out.printf("Variante %d : %d ms%n", v, (System.nanoTime() - debut) / 1_000_000);
```

7 Pour aller plus loin (facultatif) : pagination

Exercice 6 — `OFFSET` ou pagination par clé

Comparez les plans et les temps de :

```sql
SELECT * FROM Operation ORDER BY numOperation LIMIT 20 OFFSET 900000;
SELECT * FROM Operation WHERE numOperation > 900000 ORDER BY numOperation LIMIT 20;
```

Ajoutez à `BanqueDAO` une méthode `List<Operation> pageOperations(int apresNum, int taille)` qui utilise la seconde forme.

8 Rendu

— Le fichier `OPTIMISATION.md` avec vos mesures et vos explications.
— Un script `index.sql` contenant les index que vous recommandez pour l'application bancaire, chacun justifié par une requête de l'application (en commentaire).
— La classe `BenchInsertions`.